



یادگیری عمیق

نیم سال دوم ۰۳-۰۴

مدرس: مهدیه سلیمانی

تمرین سوم

ددلاین تمرین: تمرین تئوری: ۱۰ اردیبهشت ، تمرین عملی: ۱۹ اردیبهشت

- برای ارسال هر تمرین تا ساعت ۲۳:۵۹ روز ددلاین فرصت دارید. دقت شود که تمرین تئوری سوم تاخیر مجاز ندارد (به دلیل نزدیکی با میانترم درس) اما برای تمرین عملی یک هفته تاخیر مجاز دارید (یعنی حداکثر تاریخ ارسال تمرین تئوری ۱۰ اردیبهشت و تمرین عملی ۲۶ اردیبهشت است).
- در هر کدام از سوالات، اگر از منابع خارجی استفاده کرده‌اید باید آن را ذکر کنید. در صورت همفکری با افراد دیگر هم باید نام ایشان را در سوال مورد نظر ذکر نمایید.
- پاسخ تمرین باید ماحصل دانسته‌های خود شما باشد. در صورت رعایت این موضوع، استفاده از ابزارهای هوش مصنوعی با ذکر نحوه و مصداق استفاده بلامانع است.
- پاسخ ارسالی واضح و خوانا باشد. در غیر این صورت ممکن است منجر به از دست دادن نمره شود.
- پاسخ ارسالی باید توسط خود شما نوشته شده باشد. به اسکرین‌شات از منابع یا پاسخ افراد دیگر نمره‌ای تعلق نمی‌گیرد.
- در صورتی که بخشی از سوال‌ها را جای دیگری آپلود کرده و لینک آن را قرار داده باشید، حتما باید تاریخ آپلود مشخص و قابل اتکا باشد.
- محل بارگذاری سوالات نظری و عملی در هر تمرین مجزا خواهد بود. به منظور بارگذاری بایستی تمرین تئوری در یک فایل pdf با نام `HW3_[First-Name]_[Last-Name]_[Student-Id].pdf` و تمرین عملی نیز در یک فایل مجزای زیپ با نام `HW3_[First-Name]_[Last-Name]_[Student-Id].zip` بارگذاری شوند.
- در صورت وجود هرگونه ابهام یا مشکل، در کوثرای درس آن مشکل را بیان کنید و از پیغام دادن مستقیم به دستیاران آموزشی خودداری کنید.
- طراحان این تمرین: محمد امانلو، آرش رسولی، مهرداد صالحی، محمد جواد رنجبر، شایگان ادیم، امیرمحمد ایزدی

بخش نظری (۱۰۰ نمره)

پرسش اول (۱۵ نمره)

در این سوال به بررسی Backpropagation در شبکه‌های LSTM می‌پردازیم. معماری استاندارد LSTM را معرفی کرده و فرآیند کامل پس انتشار خطا (Backpropagation) آن را به صورت ریاضیاتی استخراج کنید. گام به گام مشتقات را با تمامی معادلات مرتبط ارائه دهید.

نمادگذاری متغیرها:

- x_t : ورودی در گام زمانی t
- h_t : حالت پنهان در گام زمانی t
- c_t : حالت سلول (حافظه) در گام زمانی t

• f_t : خروجی دروازه فراموشی در گام زمانی t

• i_t : خروجی دروازه ورودی در گام زمانی t

• o_t : خروجی دروازه خروجی در گام زمانی t

• g_t : کاندیدای حالت سلول در گام زمانی t

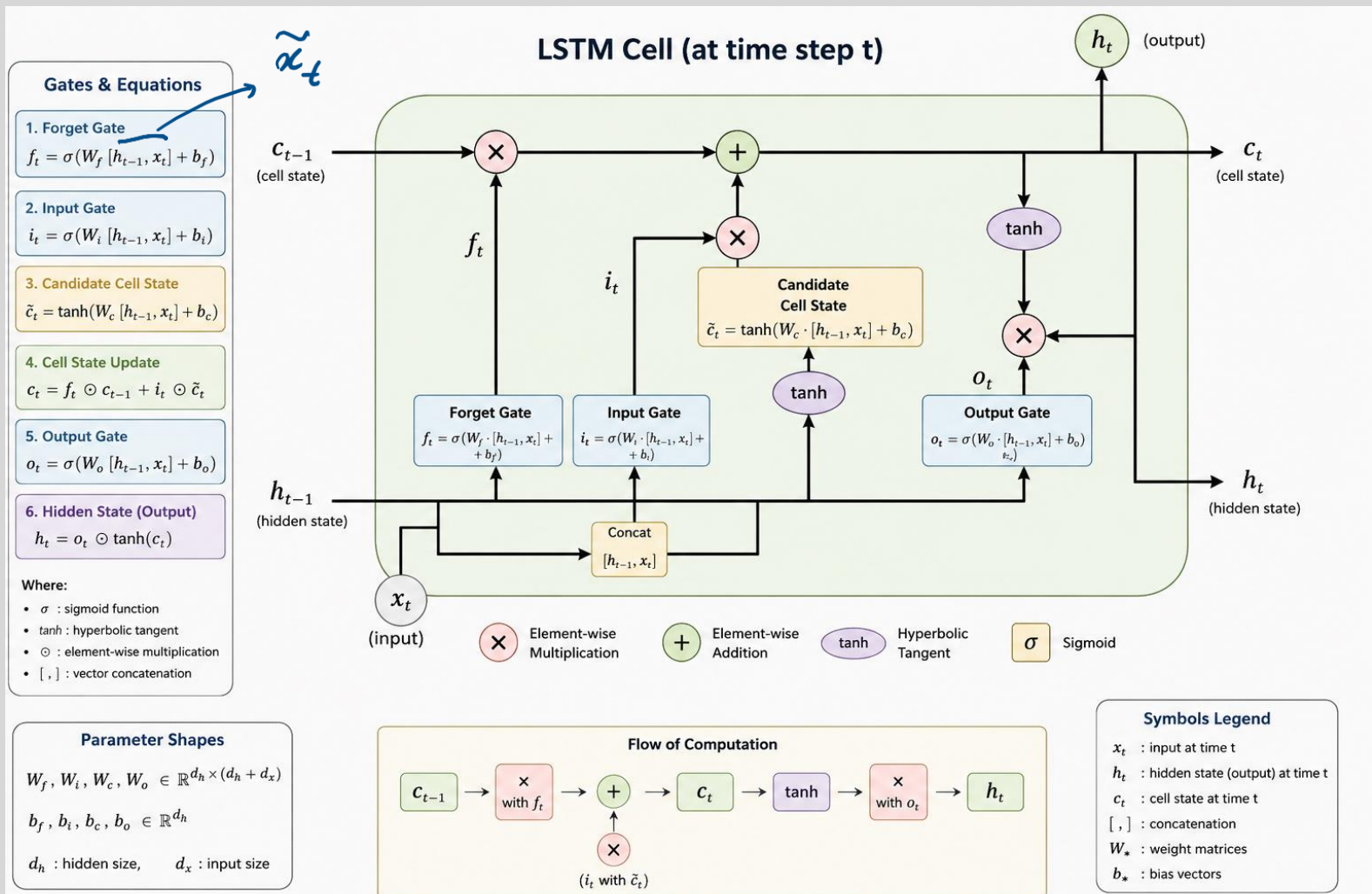
• $\tilde{x}_t = \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix}$: ترکیب حالت پنهان قبلی و ورودی فعلی، یعنی

• W_f, W_i, W_g, W_o : ماتریس‌های وزن مربوط به دروازه‌های فراموشی، ورودی، کاندیدای حالت سلول و خروجی

• b_f, b_i, b_g, b_o : بایاس‌های مربوط به دروازه‌های مذکور

معماری LSTM برای حل مشکلاتی که RNN ها داشتند به وجود آمد، توی این ساختار دیگه اطلاعات مهم با طولانی تر شدن جملات فراموش نمی شدند و از طرفی اطلاعات غیر ضروری نیز فراموش میشدن، همچنین مشکل Vanishing Gradient هم توی این ساختار بهبود پیدا کردش.

برای بخش خود معماری بنظم بهترین کار نشون دادش، به این شکل:



Gates & Equations

1. Forget Gate

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

2. Input Gate

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

3. Candidate Cell State

$$\tilde{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$

4. Cell State Update

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

5. Output Gate

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

6. Hidden State (Output)

$$h_t = o_t \odot \tanh(c_t)$$

Where:

- σ : sigmoid function
- $\tanh$: hyperbolic tangent
- $\odot$: element-wise multiplication
- $[,]$: vector concatenation

که خوب بنظر نمی آید، ولی برای قسمت گرایان باید شروع کنیم به حل کردن:

$$Loss = \frac{\partial L}{\partial h_t} \times \frac{\partial h_t}{\partial o_t}$$

گام اول

$$h_t = o_t \odot \tanh(c_t) \rightarrow \delta_h \odot \tanh(c_t)$$

$$Loss = \frac{\partial L}{\partial h_t} \times \frac{\partial h_t}{\partial o_t} \times \frac{\partial o_t}{\partial a_o} \times \frac{\partial a_o}{\partial \tilde{c}_t}$$

گام دوم

$$\delta_h \odot \tanh(c_t) \odot o_t (1 - o_t) \odot w_o$$

$$\delta_c = o_t (1 - o_t)$$

این از output gate

حالا؛ Cell State

$$h_t = o_t \odot \tanh(c_t)$$

$$\frac{\partial L}{\partial c_t} = \frac{\partial L}{\partial h_t} \times \frac{\partial h_t}{\partial c_t}$$

بی نهایت

$$\frac{\partial L}{\partial c_t} = \delta_h \odot o_t (1 - \tanh^2(c_t))$$

$$\frac{\partial h_t}{\partial c_t} = o_t (1 - \tanh^2(c_t))$$

مسئله $\delta_c^{(1)}$

نقد LSTM، برابر Backprop، c_t ؟ بعضی ها میگویند سی پی باید

$$c_{t+1} = f_{t+1} \odot c_t + i_{t+1} \odot \tilde{c}_{t+1}$$

فکر کنید

$$\frac{\partial L}{\partial c_t} = \frac{\partial L}{\partial c_{t+1}} \times \frac{\partial c_{t+1}}{\partial c_t}$$

$\delta_c^{(2)}$

δ_c^{next}

f_{t+1}

فقط در این مورد، مسیر هم به لستود

$$\delta_c^{total} = \delta_c^{(1)} + \delta_c^{(2)} = \delta_h \odot \delta_t \odot (1 - \tanh^2(c_t)) + \delta_c^{next} \odot f_{t+1}$$

f_t برابر : $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \Rightarrow \frac{\partial c_t}{\partial f_t} = c_{t-1}$

از طرفی : $\frac{\partial L}{\partial f_t} = \frac{\partial L}{\partial c_t} \times \frac{\partial c_t}{\partial f_t} = \delta_c \odot c_{t-1}$

$$\frac{\partial L}{\partial w_f} = \frac{\partial L}{\partial c_t} \times \frac{\partial c_t}{\partial f_t} \times \frac{\partial f_t}{\partial a_f} \times \frac{\partial a_f}{\partial w_f} \quad \leftarrow a_f = w_f \tilde{x}_t + b_f$$

$\delta_c \odot c_{t-1}$ $\downarrow$ $f_t(1-f_t)$ $\downarrow$ $\tilde{x}_t$

$$\frac{\partial L}{\partial b_f} = \text{"} \times \left(\frac{\partial a_f}{\partial b_f} \right) = \delta_c \odot c_{t-1} \times f_t \times (1-f_t)$$

در اینجا هم مسیر برابر i_t و $\tilde{g}_t$ هم نه از لستود، فقط $\frac{\partial \tilde{g}_t}{\partial a_g} (1 - \tanh^2(a_g))$ مسیر

پرسش دوم (۲۰ نمره)

در این سوال به بررسی ویژگی‌های تابع خودتوجه و عملکرد آن در مدل‌های ترانسفورمر می‌پردازیم.

۱. بخش الف نشان دهید تابع خودتوجه که به صورت زیر تعریف می‌شود:

$$Y = \text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{D_k}} \right) V$$

به صورت یک ماتریس تمام متصل (fully connected) در قالب یک ماتریس که تمام دنباله ورودی از بردارهای کلمه به صورت پیوسته را به یک بردار خروجی با همان بعد نگاشت می‌کند، می‌تواند توسعه یابد.

$$Q = W_Q X \quad \text{و} \quad K = W_K X \quad , \quad V = W_V X$$

در ابتدا بخش QK^T رو اگر بخوایم توضیح بدیم، از ضرب دو ماتریس Q, K به این صورت یک ماتریس مربعی ایجاد می‌شود که بیانگر Similarity هستش که میزان توجه توکن i رو نسبت به j میسنجه:

$$QK^T = \begin{bmatrix} q_1 k_1 & q_1 k_2 & \dots & q_1 k_j \\ q_2 k_1 & q_2 k_2 & \dots & q_2 k_j \\ \vdots & \vdots & \ddots & \vdots \\ q_n k_1 & q_n k_2 & \dots & q_n k_j \end{bmatrix}$$

مرحله ی بعد تقسیم این ماتریس بر $\sqrt{D_k}$ جهت نرمالسازی ورودی به تابع softmax هستش، این تابع برای مقادیری که دور از هم افتادند اگر فاصله زیاد باشد مقدار اصلی را یک و بقیه به صفر میل می‌کنند، پس از اعمال تابع، مقادیر بین صفر و یک می‌افتند، بعد از آن ضرب در بردار Values (V) میشوند، تا اینجا ما وزندهی توجه به هر یک از مقادیر رو داریم. به مدل داریم می‌گیم "با توجه به اهمیت هر توکن، یک ترکیب وزنی از اطلاعات Value ها بساز.".

اگر به فرآیند بالا نگاه کنیم و با فرآیند محاسبات fully connected مقایسه کنیم، توی حالت fully connedted بعد از آموزش، تمامی وزن ها ثابت میماند ولی برای Attention وابسته به Key بردار ورودی، مقادیر Query و میزان توجه تغییر خواهد کرد.

۲. بخش ب: سپس ثابت کنید که چنین ماتریسی از $(N^2 D^2)$ پارامتر برخوردار خواهد بود.

مقادیر توکها

اعداد توکها $\rightarrow N \times D$
 $X \in \mathbb{R}^{N \times D}$

$W_k, W_Q \in \mathbb{R}^{D \times D}$

$QK^T = (N \times D)(D \times N) = N^2 D$

مقادیر Q و K و V
 $Q = W X = (D \times D)(N \times D) \in \mathbb{R}^{N \times D^2}$

ماتریسی از Rank $N \times N$
 استود که به هر طایفه D
 مصالحه دلی

$\Rightarrow$ مجموع $= N^2 D + N D^2$
 $= O(N^2 D^2)$

۳. بخش ج: نشان دهید که شبکه خودتوجه متناظر با یک نسخه پراکنده (sparse) از این ماتریس با اشتراک گذاری پارامترها است. یک نمودار از ساختار این ماتریس را بکشید و نشان دهید که کدام بلوک‌های پارامتر به اشتراک گذاشته می‌شوند و کدام بلوک‌ها تمام عناصرشان صفر است.

در مثال قبلی با افزایش N ، سرعت محاسبه افزایش می‌یابد، پس خوب نیست که مقادیر N بسیار بزرگ را در نظر بگیریم. $O(N^2 D^2)$ و $O(N^2 D)$ در نظر بگیریم. به این حالت می‌توانیم N ها، k ها، $N \ll k$ را در نظر بگیریم و به هر طریقی که می‌خواهیم، N ها، k ها، $N \ll k$ را در نظر بگیریم و به هر طریقی که می‌خواهیم، N ها، k ها، $N \ll k$ را در نظر بگیریم.

$A = \begin{bmatrix} 0.5 & 0.5 & 0 & 0 & \dots \\ 0.2 & 0.8 & 0 & 0 & \dots \\ 0 & 0.3 & 0.7 & 0 & \dots \\ \vdots & 0 & 0.4 & 0.6 & \dots \end{bmatrix}$

از این فرآیند برای C دوم محاسبه صحت قبل

$QK^T = NkD$
 $Q = ND^2$
 or $K = V$

$NkD + ND^2 = O(N^2 D^2)$
 $k \ll N$

۴. بخش ۵: فرض کنید که $E \in \mathbb{R}^{n \times d_{\text{model}}}$ ماتریسی است که شامل بردارهای موقعیتی E_t است که موقعیت t را در یک دنباله ورودی به طول n نشان می‌دهد. به عبارت دیگر، این ماتریس تابعی از $e: \{1, \dots, n\} \rightarrow \mathbb{R}^{d_{\text{model}}}$ است که به صورت زیر تعریف می‌شود:

$$e(t) = E_{t,:} := \begin{bmatrix} \sin\left(\frac{t}{f_1}\right) \\ \cos\left(\frac{t}{f_2}\right) \\ \sin\left(\frac{t}{f_3}\right) \\ \cos\left(\frac{t}{f_4}\right) \\ \vdots \\ \sin\left(\frac{t}{f_{d_{\text{model}}}}\right) \\ \cos\left(\frac{t}{f_{d_{\text{model}}}}\right) \end{bmatrix}$$

که در آن فرکانس‌ها به صورت زیر تعریف می‌شوند:

$$f_m = \frac{1}{\lambda_m} := 10000 \frac{2m}{d_{\text{model}}}.$$

سپس نشان دهید که یک تبدیل خطی $T^{(k)} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ وجود دارد که برای آن رابطه زیر برقرار است:

$$T^{(k)} E_{t,:} = E_{t+k,:}$$

به این معنی که این تبدیل خطی برای هر انحراف موقعیتی $k \in \{1, \dots, n\}$ در هر موقعیت معتبر $t \in \{1, \dots, n-k\}$ در دنباله برقرار است.

با فرض $a = \frac{t}{f}$ و $b = \frac{k}{f}$ داریم:

$$E_{t+k} = \sin(a+b) = \sin\left(\frac{t+k}{f}\right)$$

$$\sin(a+b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

$$\sin\left(\frac{t+k}{f}\right) = \sin\left(\frac{t}{f}\right)\cos\left(\frac{k}{f}\right) + \cos\left(\frac{t}{f}\right)\sin\left(\frac{k}{f}\right) = \underbrace{\begin{bmatrix} \cos(k/f) & \sin(k/f) \end{bmatrix}}_{T^{(k)}} \underbrace{\begin{bmatrix} \sin(t/f) \\ \cos(t/f) \end{bmatrix}}_{E_t}$$

ماتریس E_t بردار موقعیتی

برای دوم هم به همین صورت

$$\cos(a+b) = \cos(a)\cos(b) - \sin(a)\sin(b) = \begin{bmatrix} -\sin(k/f) & \cos(k/f) \end{bmatrix} \begin{bmatrix} \sin(t/f) \\ \cos(t/f) \end{bmatrix}$$

$\frac{t}{f}$ $\frac{k}{f}$ $\frac{t}{f}$ $\frac{k}{f}$

هر مقدار، این جمع کسینوس با فرکانس متفاوت

BERT $d=768$ مدل

مثلاً برای مدل BERT 768

$$E_1 = [0.84, 0.54, 0.91, \dots]$$

نکته: توانمنداشت و مثلث 24.6، 25 و 26 از معادله مسکنه ورود با محاسبه با اینه ،
 و رسانت خروجی از تابع softmax ، رسانت خروجی به دست می آید و در نتیجه در خروجی بردار
 خواهد بود ، و چون خاصیت خاص با این توکونیا مشترک خاص خواهد داشت .

(۱۷)

مهرت که بردار شیب برابر $\frac{1}{2}(v_a + v_b)$ و چون خاصیت به بقای قالب است ، آنگاه معکوس آن به آینه
 و آینه ، مر لستد این است به آینه بالشت ، معکوس می به دست می آید و در نتیجه در خروجی بردار
 به مولفه ها خواهد بود .

(ب) قابلیت ترکیب: یک مدل ترنسفورمر که فقط بر یک بردار v_j متمرکز است، ممکن است بخواهد اطلاعات را از چندین بردار منبع ترکیب کند. فرض کنید که می خواهیم اطلاعات دو بردار v_a و v_b را با بردارهای کلید k_a و k_b ترکیب کنیم.

(i) چگونه باید دو بردار v_a و v_b را در یک بردار خروجی ترکیب کنیم تا اطلاعات هر دو بردار حفظ شود؟ یکی از روش های رایج برای انجام این کار در یادگیری ماشین، محاسبه میانگین است:

$$c = \frac{1}{2}(v_a + v_b)$$

استخراج اطلاعات در مورد بردارهای اصلی v_a و v_b از حاصل ممکن است به نظر دشوار بیاید، اما در شرایط خاصی می توان این کار را انجام داد. در این مسئله، خواهیم دید که جز این موارد است.

فرض کنید که اگرچه ما v_a یا v_b را نمی دانیم، اما می دانیم که v_a در یک زیرفضای A قرار دارد که توسط m بردار پایه $\{a_1, a_2, \dots, a_m\}$ تشکیل شده است، در حالی که v_b در یک زیرفضای ناهمپوشان B قرار دارد که توسط بردارهای پایه $\{b_1, b_2, \dots, b_p\}$ تشکیل شده است. تمام بردارهای پایه دارای طول ۱ و متعامد با یکدیگر هستند. علاوه بر این، فرض کنید که دو زیرفضا متعامد هستند، یعنی:

unit orthonormal

$$\langle a_i, b_k \rangle = 0 \quad \text{همه } i \text{ و } k.$$

اگر $v_a \in A$ و $v_b \in B$ ، می توانیم از ماتریس M برای استخراج v_a از بردار مجموع $s = v_a + v_b$ استفاده کنیم. به عبارت دیگر، می خواهیم M را طوری بسازیم که برای هر v_a :

ماتریس متعامد

$$M \cdot s = v_a.$$

نکته: v_a و v_b باید به صورت یک بردار در $\mathbb{R}^d$ بیان شوند، نه بر حسب بردارهای A و B .
 نکته: با توجه به اینکه بردارهای $\{a_1, a_2, \dots, a_m\}$ هم متعامد هستند و هم مبنای نرمال شده ای برای v_a می دانیم که مقدار $\{c_1, c_2, \dots, c_m\}$ وجود دارد به طوری که $v_a = (c_1 a_1 + c_2 a_2 + \dots + c_m a_m)$
 (ii) همانطور که قبلاً گفتیم، تصور کنید v_a و v_b دو بردار مقدار باشند که به ترتیب مربوط به بردارهای کلید k_i هستند. فرض کنید که تمام بردارهای کلید متعامد هستند، بنابراین $\langle k_i, k_j \rangle = 0$ برای همه $i \neq j$ و تمام بردارهای کلید دارای طول ۱ هستند. یک عبارت برای بردار پرس و جو q پیدا کنید به طوری که:

$$\frac{1}{2}(v_a + v_b) \approx c.$$

نقشه ماتریس Projection برابر بردار $(\vec{v}_a, \vec{v}_b) \Rightarrow S = \vec{v}_a + \vec{v}_b$ (i)

مورد دیگر شکل $M.S = \vec{v}_a$ ، $\vec{v}_a$ این حفظ بردارهایی هستند که $\vec{v}_a$ و $\vec{v}_b$ هستند
 توسط زیرفضای تولید شده باشد که $A \perp B = \langle \vec{v}_a, \vec{v}_b \rangle = 0$ if $\vec{v}_a \in A$ و $\vec{v}_b \in B \Rightarrow$

(ii)؟ صحت کل این صحت بیشتر بر معنای بردار دارد، همانطور که بردار قبلی بودند،
 در مورد Attention $\vec{v}_a$ بردار، جامع $\vec{v}_a$ و $\vec{v}_b$ هر دو را از خود داخل می‌گیرد
 یاها را همزاده و ماتریس $\vec{v}_a$ و $\vec{v}_b$ می‌باشد؟ صحت میانگین در $\vec{v}_a$ و $\vec{v}_b$ است، اینو بخش لستون
 دقیقاً همان باز آن سؤال می‌باشد.

(ج) رفع یکی از اشکالات توجه تک سر: در قسمت قبل دیدیم که چگونه ممکن است توجه تک سر به طور مساوی روی دو مقدار متمرکز شود. این مفهوم را می‌توان به راحتی به هر زیرمجموعه‌ای از بردارهای مقدار تعمیم داد. در این سوال خواهیم دید که چرا این راه حل عملی نیست.

مجموعه‌ای از بردارهای کلید $k_1, \dots, k_n$ را در نظر بگیرید که اکنون به صورت تصادفی نمونه برداری شده‌اند:

$$k_i \sim N(\mu_i, \Sigma_i),$$

که در آن میانگین μ_i برای شما شناخته شده است، اما کوواریانس Σ_i ناشناخته است. علاوه بر این، فرض کنید:

$$\|\mu_i\| = 1,$$

و بردارهای میانگین μ_i همگی متعامد هستند. اگر $j \neq i$ ، آنگاه:

$$\langle \mu_i, \mu_j \rangle = 0.$$

(i) تصور کنید که ماتریس‌های کوواریانس برای α بسیار کوچک به صورت $\Sigma_i = \alpha I$ ، برای $\forall i \in \{1, \dots, n\}$ تعریف می‌شوند. یک پرس و جو q را بر حسب μ_i طراحی کنید که k_i ها از توزیع نرمالی با میانگین آن‌ها نمونه برداری شوند، طوری که مانند قبل:

$$c \approx \frac{1}{2}(v_a + v_b).$$

یک استدلال مختصر در مورد اینکه چرا این اتفاق می‌افتد ارائه دهید.

نمونه‌ای که μ_i می‌تواند باشند، آن‌ها $\vec{v}_a$ و $\vec{v}_b$ هستند. این دو بردار همگن و هم‌دگر و متعامد هستند.
 بردار خود میانگین هستند، پس $\vec{v}_a$ و $\vec{v}_b$ را نیز تولید می‌کنند.
 و بردارهای متعامد μ_i و μ_j متعامد هستند. $\vec{v}_a$ و $\vec{v}_b$ نیز در نتیجه متعامد هستند.

$\vec{v}_a$ و $\vec{v}_b$ متعامد هستند
 $\vec{v}_a$ و $\vec{v}_b$ متعامد هستند
 $\vec{v}_a$ و $\vec{v}_b$ متعامد هستند

بردارهای k_a متمرکز می‌شوند
 $k_a^T \approx \frac{1}{2}(\bar{v}_a + \bar{v}_b)$

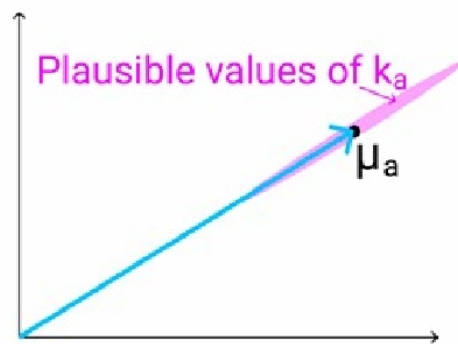
(ii) اگرچه توجه تک‌سر در برابر آشفتگی‌های کوچک در کلیدها مقاوم است، اما برخی از انواع آشفتگی‌های بزرگ‌تر ممکن است مشکلات جدی‌تری ایجاد کنند. به طور خاص، در برخی موارد، یکی از بردارهای کلید k_a ممکن است از نظر نرم بزرگ‌تر یا کوچک‌تر از بقیه باشد، در حالی که همچنان در همان راستای میانگین μ_a است. به عنوان مثال، تصور کنید که کوواریانس نمونه بسیار کوچک a به صورت زیر باشد:

$$\Sigma_a = \frac{1}{2}(\mu_a \mu_a^T) + \alpha I$$

در نظر بگیرید این باعث می‌شود که k_a تقریباً در جهتی مشابه μ_a باشد، اما با واریانس‌های بزرگ در اندازه. علاوه بر این، در نظر بگیرید که:

$$\Sigma_a = \alpha I$$

برای همه $i \neq a$.



شکل ۳: بردار μ_i (که در اینجا به صورت دوبعدی به عنوان مثال نشان داده شده است)، با محدوده‌ی مقادیر ممکن k_a به رنگ بنفش نشان داده شده است. همانطور که قبلاً ذکر شد، k_a تقریباً در جهتی مشابه μ_a است، اما ممکن است اندازه‌ی آن بزرگ‌تر یا کوچک‌تر باشد.

وقتی چندین بار از $\{k_1, \dots, k_n\}$ نمونه‌برداری می‌کنید و از بردار q که در قسمت i تعریف کردید استفاده می‌کنید، انتظار دارید بردار c از نظر کیفی برای نمونه‌های مختلف چگونه باشد و مقدار آن در چه حدود مقادیری تغییر کند؟

چون بردار k_a همچنان تقریباً در راستای μ_a قرار دارد، Query همچنان بیشترین شباهت را با آن خواهد داشت. اما به دلیل واریانس زیاد در راستای μ_a ، اندازه‌ی k_a در نمونه‌های مختلف تغییر می‌کند. این تغییر باعث می‌شود مقدار qk_a^T و در نتیجه خروجی تابع Softmax نوسان قابل توجهی داشته باشد. بنابراین وزن Attention مربوط به v_a در اجراهای مختلف ثابت نخواهد بود و بردار خروجی c نیز دچار تغییر و ناپایداری می‌شود. به عبارتی "جهت (Direction) بردار تقریباً حفظ می‌شود، اما اندازه (Norm یا Magnitude) آن تغییر می‌کند".

از نظر کیفی، پاسخ‌هایی که مدل ارائه می‌دهند پایداری کمتری دارند، یعنی ممکنه در همان کلاس (برای semantic task) یک بار Positive و یک بار بکره و یک بار Neutral ولی خب هیچوقت (یعنی با احتمال خیلی کم) نمیداد بگه Awful.

(د) راه حل با استفاده از مزایای توجه چندسره: اکنون می‌خواهیم در مورد توانایی خودتوجهی چندسره در مواجهه با این مشکل صحبت کنیم. ما یک نسخه ساده از خودتوجهی چندسره را در نظر خواهیم گرفت که مشابه خودتوجهی تک‌سره است که در این تمرین ارائه کرده‌ایم، به جز اینکه دو بردار پرس و جو q_1, q_2 تعریف شده است که منجر به یک جفت از بردارهای c_1, c_2 می‌شود که هر یک خروجی خودتوجهی تک‌سره نسبت به بردار پرس و جو مربوطه خود هستند. خروجی نهایی خودتوجهی چندسره، میانگین آن‌هاست:

$$\frac{1}{2}(c_1 + c_2).$$

همانطور که در بخش سوم این سوال، مجموعه‌ای از بردارهای کلید $\{k_1, \dots, k_n\}$ را در نظر گرفتیم که به طور تصادفی نمونه‌برداری می‌شوند:

$$k_i \sim N(\mu_i, \Sigma_i),$$

که در آن میانگین μ_i برای شما شناخته شده است، اما کوواریانس Σ_i ناشناخته است. فرض می‌کنیم که میانگین μ_i متعامد هستند و برای $i \neq j$:

$$\langle \mu_i, \mu_j \rangle = 0.$$

همچنین نرم واحد دارند:

$$\|\mu_i\| = 1.$$

(i) فرض کنید که ماتریس‌های کوواریانس برای α های بسیار کوچک به صورت $\Sigma_i = \alpha I$ تعریف شده‌اند. بردارهای q_1 و q_2 را به گونه‌ای پیدا کنید که خروجی برابر با میانگین دو بردار v_a و v_b شود.

(ii) فرض کنید که ماتریس‌های کوواریانس برای α های بسیار کوچک به صورت:

$$\Sigma_a = \frac{1}{2}(\mu_a \mu_a^T) + I$$

و برای هر i ، اگر $i \neq a$ ، آنگاه برابر:

$$\Sigma_i = \alpha I$$

هستند. بردارهای پرس و جو q_1 و q_2 را که در بخش قبلی سوال طراحی شده‌اند، در نظر بگیرید. انتظار دارید خروجی c بر اساس نمونه‌های مختلف بردارهای کلید چگونه باشد؟ یا به عبارت دیگر، چه رابطه‌ای بین بردار c و بردارهای مقدار مربوط به کلیدها وجود خواهد داشت؟ (لطفاً به طور خلاصه توضیح دهید که چرا می‌توانید مواردی را که در آن $k_a^T q_i < 0$ است نادیده بگیرید.)

در حالت چندسره، هر Head می‌تواند روی یک زیرمجموعه از اطلاعات تمرکز کند. بنابراین می‌توان q_1 را برای بازیابی v_a و q_2 را برای بازیابی v_b طراحی کرد، به طوری که $c_1 \approx v_a$ و $c_2 \approx v_b$ شود. خروجی نهایی برابر میانگین دو Head است و تقریباً برابر $\frac{1}{2}(v_a + v_b)$ خواهد بود. در حالتی که یکی از Key ها دارای واریانس بزرگ باشد، نوسان آن فقط روی Head متناظر اثر می‌گذارد و Head دیگر همچنان پایدار باقی می‌ماند. بنابراین میانگین خروجی دو Head نسبت به Self-Attention تک‌سره پایدارتر بوده و حساسیت کمتری به نویز و تغییرات تصادفی Key ها خواهد داشت.

پرسش چهارم (۲۰ نمره)

یکی از مشکلات عملیاتی مکانیزم توجه با context window های بزرگ، هزینه‌ی محاسباتی بسیار زیاد محاسبه‌ی ماتریس توجه است. برای رفع این مشکل، روش‌های بسیاری به کار گرفته شده که مهم‌ترین آن‌ها، روش‌های KV-Cache هستند. اما این روش‌ها نیز مشکل مخصوص به خود را دارند که استفاده‌ی خیلی زیاد از حافظه‌ی cache است.

در این تمرین، قصد داریم هزینه محاسباتی یک گام inference تک توکنی از یک لایه Multi-head Attention در یک مدل زبانی بزرگ decoder-only را تحلیل کنیم و نقاط قوت و ضعف اعمال روش‌های KV-Cache را بررسی کنیم.

فرضیات:

Base of GPT

• n : تعداد توکن‌ها

• d : بعد Embedding ورودی هر توکن

• d_h : بعد فضای Key-Query

• n_h : تعداد سرهای Attention

۱. تحلیلی دقیق از هزینه محاسباتی یک لایه از این ساختار را بر حسب متغیرهای داده شده ارائه دهید. توجه داشته باشید که در این بخش باید فرض کنید که هیچ روش بهینه‌سازی‌ای برای بهبود مکانیزم توجه نداریم.

سایر باید مثل عدد بیستم بیست باشد

$n = 3$ $d = 2$ $\Rightarrow$ $Q = X W_q$ $K = X W_k$ $Value = X W_v$

I Love AI

$W_q = \begin{bmatrix} & \\ & \end{bmatrix}_{2 \times 2}$ و $X = \begin{bmatrix} & \\ & \end{bmatrix}_{3 \times 2}$ $\Rightarrow Q = \begin{bmatrix} & \\ & \end{bmatrix}_{3 \times 2}$

دفعه‌های ابعاد 3×2 $\rightarrow$ Q و K و V هم هست

مدرسه QK^T و در هر یک در یک ماتریس 3×3 ، بعد از آن قسم به $\sqrt{d}$ در هر یک

ماتریس softmax و بعد از آن ضرب V $\leftarrow$ $\begin{bmatrix} & \\ & \end{bmatrix}_{3 \times 2} \times \begin{bmatrix} & \\ & \end{bmatrix}_{3 \times 3}$ matrix

برای حالت Multi-head و اینکه این کار چقدر ساده

قسم به تعداد head ها شده، مثلاً برای $d = 816$ $\Rightarrow$ 12 تا head ها شود

هر head $= \frac{816}{12} = 68$ تا d

در وادی بالا جا به جا شود.

$Q = (n \times d)(d \times d_h) = n d d_h$ ؟ قیمت مقادیر
 هر سر هر یک از head هر سر به ماتریس

$3 n d d_h \leftarrow Q, K, V$ که W و K و Q
 $3 n_h n d d_h \leftarrow$ هر سر n_h head
 $n_h d_h = d \Rightarrow 3 n d^2$ که هر سر d به d به ماتریس

$(n \times d_h)(d_h \times n) \Rightarrow n^2 d_h$ هر سر آن به هر سر $Q K^T$

$n_h n d_h = \text{Multihead Attention}$

$O(n^2 d)$ باز هم d که
 $n_h d_h = d$

$\text{Softmax}\left(\frac{Q K^T}{\sqrt{d_h}} + M\right)$
 $O(n_h n^2)$

ترکیب: Softmax, Mask, Scaling $\leftarrow$

$A \bar{V} = n_h n^2 d_h$
 $\text{Softmax}(\sim)$ Score
 $C_{A \bar{V}} = n^2 d$

ترکیب Value $\rightarrow$

برای Softmax دایره $n \times n$ و یک سبک و هکس سطر عدد ورودی
 $O(n_h n)$
 multihhead

برای $A = 1 \times n$ و $V = n \times d_h$: head
 $O(1 \times n \times d_h)$
 multihhead $\rightarrow O(1 \times n_h \times n \times d_h)$

برای کل : $3n_h d_h d_h + 2n_h n d_h + n d_h d_h + O(n_h n)$
 $n_h \ll n$

$\hookrightarrow$ if $n_h d_h = d \Rightarrow \text{Total cost} = 4d^2 + 2nd + O(n)$

برای حافظه مورد نیاز ذخیره : Cache

key: d_h Value: $d_h \xrightarrow[\text{single head}]{2nd_h} \text{Multihhead} = 2n_h n d_h$

۳. ابتدا روش Multi-Query Attention را کامل توضیح دهید و بار دیگر، همان تحلیل را با استفاده از KV-cache معمول برای ماتریس‌های کلید و مقدار انجام دهید، اما این بار در معماری مدل از Attention Multi-Query استفاده می‌کنیم. همینطور مانند بخش قبل، تعداد عناصر (اعداد) مورد نیاز برای ذخیره در حافظه را محاسبه کنید.

در این مرحله، ما به بار در نظر گرفتن key و value هر منفی برای head هر متفاوت، و آنگاه هر بار
 و این که هر بار متفاوت داریم، پس اگر متوجه شدیم که باید برای هر $n_h n d_h$ مورد نیاز، اما
 $n d_h$ برای حالت بدون KV-cache
 مورد نیاز k و v

والتكache كدليم مستورد $2dd_h$ ، سى كى $Projection$ كا لستون $d^2 + 2dd_h$ وبلر dd
 $d^2 + \frac{2d^2}{n_h}$
 $d = n_h d_h$

در نهایت داریم:
$$2d^2 + \frac{2d^2}{h} + 2nd$$

$$\downarrow$$

$$\text{softmax}(\sim) \times \bar{V}$$

$$\leftarrow \text{ناتمامی}$$

کامرس حافلہ

→ حاله
2nd
n_h

که درستی که منی حاصله کنم را بگذرد و ولی تنوع و بلشت اینجی سر نمید
 اینجی معادله باز هم با اینک معادله السوال شده و بلشت کمد و لغو اولی از این معادله مقصود باشد
 حویله بیشتر در صیغه معروف از این برکت معادله می باشد اما در این معادله و در معادله attention

۴. ابتدا روش Grouped-Query Attention را کامل توضیح دهید و سپس همان تحلیل را با استفاده از KV-cache معمول برای ماتریس‌های کلید و مقدار انجام دهید، اما این بار از Grouped-Query Attention استفاده می‌کنیم. تعداد عناصر (اعداد) مورد نیاز برای ذخیره در حافظه پنهان را محاسبه کنید. (g را به عنوان تعداد گروه‌ها در نظر بگیرید.)

برای هر Q و K یک $\text{Multi-Query Attention}$ وجود دارد و نتایج V ؛ با توجه به نظر شما در نظر بگیرید.

تکثیر هر V با head ها یا بین V و V از این باب است (معمولاً در نظر گرفته می‌شود، تعداد head ها و

9. د دسته یا گروه (group) قسمتی کنیم و برابر هر گروه یک سر آتوم را در نظر بگیریم، مثلاً

$n_h = 12$ و $g = 4$ که ما 3 دسته آتوم برابر یکدیگر داریم.

→ K⁺ cahe
8 C₁ lako

d^2 برابر Q مقسوم علی کند
(مستند از ابو مقیران)

$\frac{d\phi}{dh} \propto \sqrt{h}$

$MQA \rightarrow$ روسخه

$\Rightarrow$ $Cost = d^2 + 2gd$
 $n_h d = d \Rightarrow Cost = 2d^2 + \frac{2gd^2}{n_h} + 2nd$
 $\Rightarrow d_h = \frac{d}{n_h}$

بدلر حافطه چ ما ج تا تروو دليم ، عددكلام d_h تا عددلند ،
عدد طريم و عدد بدلر n و عدد با فرض $d_h = d/n_h$
سي بدلر تروو $2nd_h$

۵. با فرض‌های زیر و با استفاده از fp۱۶، مقدار عددی پیچیدگی محاسباتی و همچنین حافظه‌ی cache استفاده شده را در هر یک از موارد بالا را محاسبه و مقایسه کنید:

$$n = 1.24 \bullet$$

$$d = \vee \wp \wedge \bullet$$

$$d_h = 94 \bullet$$

$$n_h = 12 \bullet$$

$$g = \mathfrak{f} \bullet$$

$$C_{total} = 4nn \frac{dd}{h} + 2nn^2 \frac{d}{h} + O(nh^2)$$

حالت ۱) MA میخوردی به KV و

if $n_h d_h = d \Rightarrow d_h = \frac{d}{n_h}$

$$C_{total} = 4nd^2 + 2n^2d^2 + O(\underbrace{n_h n^2})$$

$$\hookrightarrow C_{\text{total}} = 4.03 \times 10^9$$

حالت دوم: $n_h \ll n$ MHA + k-vi cache

$$3n_h d d_h + 2n_h n d_h + n d_h d + O(n_h n) \quad \text{برابر کرد}$$

if $n_h d_h = d \Rightarrow \text{Total cost} = 4d^2 + 2nd + O(n)$

بر حافظه مورد نیاز ذخیره Cache

key: d_h Value: $d_h \rightarrow \underbrace{2n_h d_h}_{\text{single head}} \rightarrow \text{Matihhead} = 2n_h n d_h$

$C_{\text{total}} = 4 \times (768)^2 + 2(1024)768 = 3932160$ C_{all}

حافظه: $2nd = 1542864$ $\xrightarrow{\text{مربوط}}$ $\sim = 3MB$
 $16 \text{ bit} \leftarrow 2 \text{ byte}$

حالت سوم: Multi-Query Attention

$$C_{\text{total}} = 2d^2 + \frac{2d^2}{n_h} + 2nd = 2(768)^2 + \frac{2(768)^2}{12} + 2^{11} \times 768 = 2850816$$

حافظه: $\frac{2nd}{n_h} = \frac{2 \times 1024 \times 768}{12} = 137072 \xrightarrow{\times 2} 274144 \text{ KB}$

حالت چهارم:

$$C_{\text{total}} = 2d^2 + \frac{2nd^2}{n_h} + 2nd = 3145728$$

حافظه: $\frac{2nd}{n_h} = 524288 \xrightarrow{\times 2} 1048576 \approx 1MB$

۶. (امتیازی) در یک نوآوری خلاقانه، در مدل DeepSeek-R1، نوع جدیدی از معماری توجه به نام Attention Multi-Head Latent به کار گرفته شد که سرعت inference را به شدت افزایش و استفاده از حافظه را به شدت کاهش داد. معماری مکانیزم توجه در این مدل را به طور کلی توضیح دهید و نشان دهید که چگونه این روش استفاده از حافظه را به شدت کم می‌کند.

در این مدل، به جای استفاده از یک هدر، از چندین هدر کوچکتر استفاده می‌شود. این هدرها به صورت V و K و Q (که در اینجا Q همان V است) استفاده می‌شوند. این هدرها به صورت V و K و Q (که در اینجا Q همان V است) استفاده می‌شوند.

فرآیند Attention با استفاده از $d_c \ll d \rightarrow c = XW_D \Rightarrow c \in \mathbb{R}^{d_c} \Rightarrow d_c \ll d$ و W ها $Transpose$ می‌شوند. V و K ها $Upscaling$ انجام می‌دهند. مانند شبکه $U-Net$ $V = cW_V^{UP}$ و $K = cW_K^{UP}$

دلیل کاهش حافظه: در Multi head latent Attention (MHLLA) به جای d از d_c استفاده می‌شود. بهر هر، فقط d ذخیره می‌شود. به جای $\frac{d_c}{d}$ کاهش می‌دهد.

پرسش پنجم (۱۵ نمره)

۱. یکی از اولین و معروف‌ترین مدل‌ها Encoder-Only مدل BERT است که در سال ۲۰۱۸ معرفی شد. در مورد این مدل به سوالات زیر پاسخ دهید.

(آ) توضیح دهید که معنی اینکه می‌گوییم مدل BERT از ترنسفورمر دو طرفه استفاده می‌کند، چیست؟

در صورت RNA ها ما به روش آموزش در طرف $Bidirectional$ آموزش می‌دهیم. ما به روش $causal$ (مانند $causal$) آموزش می‌دهیم. ما به روش $causal$ (مانند $causal$) آموزش می‌دهیم. ما به روش $causal$ (مانند $causal$) آموزش می‌دهیم.

(ب) آموزش این مدل بر اساس دو وظیفه Masked Language Modeling (MLM) و Next Sentence Prediction (NSP) بوده است. هر کدام از این وظایف را توضیح دهید و بگویید چگونه به آموزش و یادگیری مدل کمک می‌کند.

(ج) نقش توکن ویژه CLS چیست و چگونه از آن در هنگام تنظیم دقیق Fine-Tuning استفاده می‌شود؟

BERT در مرحله Pre-training با دو وظیفه اصلی آموزش داده می‌شود. در Masked Language Modeling تعدادی از توکن‌های ورودی به صورت تصادفی Mask می‌شوند و مدل باید مقدار اصلی آن‌ها را با استفاده از Context دوطرفه‌ی نزدیک بهش پیش‌بینی کند. این کار باعث یادگیری نمایش‌های معنایی و دستوری کلمات می‌شود. در Next Sentence Prediction، دو جمله به مدل داده می‌شود و مدل باید تشخیص دهد آیا جمله‌ی دوم ادامه‌ی منطقی جمله‌ی اول است یا خیر. این وظیفه باعث یادگیری ارتباط بین جملات و درک بهتر ساختار متن می‌شود.

BERT با استفاده از توکن مخصوص CLS می‌آید و در ابتدای ورودی جمله اول قرار گرفته که و بردار خروجی برای کل دو جملع تولید می‌کند که نماینده‌ی کل دو جمله می‌شود، اگر دو جمله کاملاً در ادامه همدیگر باشد، $p(Is\ Next | CLS)$ مقدار نزدیک به ۱ خواهد گرفت، وگرنه احتمال رد می‌شود.

(د) آیا می‌توان با استفاده از مدل BERT وظیفه تولید متن را انجام داد؟ توضیح دهید که چگونه می‌شود یا چرا نمی‌شود.

خیر، در این مدل، چون همونطور که گفته شد از دو طرف آموزش انجام می‌شود، یعنی مدل هم به گذشته دسترسی داشته و هم آینده را می‌بیند، بنابراین وظیفه‌ی Next Token Prediction را با تغلب انجام می‌دهد، این task معمولاً به عهده مدل‌های Decoder-Only مثل GPT ها هست.

(ه) یکی دیگر از مدل‌های موفق که پس از BERT معرفی شد مدل RoBERTa بود. توضیح دهید که چه تفاوتی در آن ایجاد شد.

RoBERTa یک نسخه‌ی بهینه‌شده از BERT است که معماری Transformer آن تقریباً مشابه BERT باقی مانده است، اما روش آموزش تغییر کرده است. مهم‌ترین تفاوت‌ها شامل حذف وظیفه‌ی Next Sentence Prediction، استفاده از Dynamic Masking در Masked Language Modeling که به صورت خلاصه، در هر Epoch توکن‌ها یا کلمات حذف شده جایگاهشان تغییر می‌کرد، آموزش با داده‌ی بیشتر، افزایش Batch Size و تعداد مراحل آموزش، و استفاده از Tokenizer مبتنی بر BPE است. این تغییرات باعث شد RoBERTa عملکرد بهتری نسبت به BERT در بسیاری از وظایف NLP داشته باشد.

(و) در مورد مدل ViT تحقیق کنید و به صورت مختصر بیان کنید که چه تفاوت و شباهتی با مدل BERT دارد.

ViT با الهام از BERT، تصویر را به صورت یک توالی از Patchها در نظر می‌گیرد و همان معماری Transformer Encoder را روی آن اعمال می‌کند. شباهت اصلی این دو مدل در استفاده از Self-Attention، Encoder Transformer، Positional Encoding و CLS Token است. تفاوت اصلی در نوع ورودی است؛ BERT با توکن‌های متنی کار می‌کند، در حالی که ViT تصویر را به Patchهای کوچک تقسیم کرده و هر Patch را مانند یک Token پردازش می‌کند. همچنین هدف آموزشی BERT یادگیری نمایش‌های زبانی با MLM و NSP است، در حالی که ViT معمولاً برای وظایف بینایی مانند Classification آموزش داده می‌شود.

ویژگی	ViT	BERT
حوزه	Computer Vision	NLP
ورودی	Patch	Token
Encoder	Transformer Encoder	Transformer Encoder
Embedding	Patch Embedding	Word Embedding
Positional Encoding	موقعیت Patchها	موقعیت کلمات
CLS Token	دارد	دارد
هدف	Classification تصویر	فهم متن

۲. در همان سال‌هایی که مدل BERT معرفی شد، مدل Generative Pre-Training Transformer (GPT) معرفی شد که یک مدل Decoder-Only به حساب می‌آمد. معرفی معماری GPT پایه و ایده اصلی‌ای شده است برای تمام مدل‌های زبانی بزرگ مانند ChatGPT که امروزه می‌بینید. در این باره به سوالات زیر پاسخ دهید.

(آ) آموزش این مدل‌ها عموماً با وظیفه Next Token Prediction (NTP) انجام می‌شوند. آن را توضیح دهید.

این روش در اسکرو مدل Masked Language Modeling (MLM) هست که با این روش که بر اساس
فقط با ریست توکن مارکتی (Decoder only) شروع و با ساخت توکن‌ها و کلمات جدید می‌کنند و این کلمات را
با این روش masked هستند که پس از مدتی می‌توان با راهنمای مثال خطای Loss مدل و Cross Entropy
به کمک معادله انتشار به Back propagation آموزش می‌دهیم.

(ب) مکانیزم Masked-Attention در مدل‌های GPT چیست و چرا برای تولید متن ضروری است؟

همانطور که در قسمت قبل هم اشاره شد، در مدل‌های GPT از Masked Attention استفاده می‌شود تا هر توکن فقط به توکن‌های قبلی خود دسترسی داشته باشد و نتواند اطلاعات توکن‌های آینده را مشاهده کند. این محدودیت با قرار دادن مقدار $-\infty$ روی بخش‌های مربوط به آینده در ماتریس Attention و قبل از Softmax اعمال می‌شود. این مکانیزم برای وظیفه‌ی Next Token Prediction ضروری است، زیرا باعث می‌شود مدل واقعاً یاد بگیرد توکن بعدی را بر اساس Context قبلی پیش‌بینی کند و از مشاهده‌ی جواب جلوگیری شود.

(ج) روش تولید متن در این معماری را توضیح دهید. یعنی توضیح دهید که ورودی مدل در ابتدا چه بوده و هر بار که لغت جدید تولید می‌شود چه اتفاقی صورت می‌گیرد.

در تولید متن توسط GPT، مدل در هر مرحله بر اساس احتمال شرطی توکن بعدی را پیش‌بینی می‌کند:

$$p(x_{i+1}|x_i, x_{i-1}, x_{i-2}, \dots)$$

یعنی با توجه به تمام توکن‌های قبلی، یک توزیع احتمال روی توکن‌های ممکن ایجاد می‌شود. سپس یک توکن بر اساس این توزیع انتخاب شده و به ورودی مدل اضافه می‌شود. با اضافه شدن هر توکن جدید، Context افزایش یافته و مدل دوباره احتمال توکن بعدی را محاسبه می‌کند تا متن به صورت خودبازگشتی (Autoregressive) تولید شود.

(د) چرا معماری‌های Decoder-Only در زمان استنتاج کندتر از معماری‌های Encoder-Only هستند؟

به دلیل همان فرآیند خودبازگشتی (Autoregressive) در این مدل‌ها، هر توکن باید مراحل ساختش با سایر توکن‌های دیگه با استفاده از توزیع احتمال شرطی ایجاد بشود.

(ه) طول کانتکست در یک مدل زبانی بزرگ چگونه بر توانایی آن در حفظ و بازیابی اطلاعات از ابتدای ورودی تأثیر می‌گذارد و چه تأثیری بر پیچیدگی محاسباتی و میزان حافظه مصرفی دارد؟

افزایش طول Context باعث می‌شود مدل اطلاعات بیشتری از متن را در اختیار داشته باشد و توانایی درک وابستگی‌های بلندمدت آن افزایش یابد، اما هزینه‌ی محاسباتی و حافظه‌ای نیز افزایش پیدا می‌کند. در Self-Attention معمولی این هزینه به دلیل وجود ماتریس QK^T با مرتبه‌ی $O(n^2d)$ رشد می‌کند، بنابراین برای مدیریت Context‌های طولانی از روش‌هایی مانند KV-Cache و انواع Attention بهینه‌شده استفاده می‌شود.

(و) چرا مدل‌های GPT ممکن است اطلاعات نادرست یا به اصطلاح "hallucination" تولید کنند، حتی اگر روی داده‌های واقعی آموزش دیده باشند؟

مدل‌های GPT به دلیل ماهیت autoregressive خود، خروجی را بر اساس توزیع احتمال شرطی توکن بعدی تولید می‌کنند:

$$p(x_{i+1}|x_i, x_{i-1}, x_{i-2}, \dots)$$

این مدل‌ها در فضای پنهان چندبعدی خود (پارامتر d) الگوهای آماری موجود در داده‌های آموزشی را یاد می‌گیرند، اما هدف آن‌ها مستقیماً مدل‌سازی حقیقت فیزیکی یا بررسی صحت اطلاعات نیست؛ بلکه تولید توالی‌هایی است که از نظر آماری مشابه داده‌های آموزشی باشند. بنابراین در شرایطی که اطلاعات کافی وجود نداشته باشد یا چند پاسخ محتمل وجود داشته باشد، مدل ممکن است متنی تولید کند که از نظر زبانی صحیح و محتمل به نظر برسد، اما با واقعیت سازگار نباشد که به آن Hallucination گفته می‌شود.

مثلاً برای یک کشور خیالی، که اصلاً وجود خارجی ندارد ممکن است نگوید که این کشور را نمی‌شناسد یا وجود ندارد، بلکه ممکن است براساس نزدیک ترین حالت ممکن را با داده‌های آموزشی به عنوان جواب برگرداند یا ممکن است یک نام شهری تولید کند که از نظر آماری شبیه پاسخ‌های قبلی باشد، ولی واقعیت نداشته باشد.

۳. یکی از روش‌های تنظیم دقیق مدل‌های بزرگ استفاده از روش‌های Parameter Efficient Fine-Tuning یا همان PEFT است که موجب صرفه‌جویی و بهینه کردن زمان و حافظه مورد نیاز برای آموزش است. حال به سوالات زیر پاسخ دهید:

(آ) توضیح دهید که روش‌های PEFT چگونه به بهینه کردن آموزش مدل‌ها کمک می‌کنند. همچنین در مورد روش‌های Adapter, Prompt-Tuning و LoRA که از این دسته از روش‌ها هستند توضیح مختصری بدهید و نحوه عملکرد آن‌ها را شرح دهید.

همانطور که در مراجع اصلی درس مشاهده کردیم، متد های Fully fine-tuning بیشترین دقت عملکردی رو روی داده های آزمون داشتند، هدف (PEFT) Parameter-Efficient Fine-Tuning این هست که با دستکاری هر چه کمتر و آموزش پارامتر های جدید، به عملکردی نزدیک به عملکرد مدل های از پیش آموزش دیده برای task های جدید گردیم، از جمله این متد ها هستش:

- Prompt Tuning: از طریق prompt ورودی و افزودن چندین پارامتر به آن، Embedding های قابل آموزش تولید شود که در فرآیند های جدید فقط آنها آموزش پیدا کنند.
- Adapter: یک MLP کوچک بین Layer های Transformer اضافه می‌شود. تمام وزن های اصلی Freeze هستند. فقط Adapter آموزش می‌بیند.
- low-rank adaption (LoRA): به جای آپدیت تمامی وزن های معماری های بسیار عظیم LLM ها برای task های جدید، یک زیر فضای بسیار با ابعاد و rank کمتری با نام ΔW آموزش داده شود که همین $\Delta W = BA$ و برای مدل نهایی $W' = W_0 + BA$ می باشد.

خلاصه:

روش های PEFT با ثابت نگه داشتن اکثر پارامترهای مدل از پیش آموزش دیده و آموزش تنها تعداد محدودی پارامتر جدید، هزینه ی Fine-Tuning مدل های بزرگ را کاهش می‌دهند. Prompt Tuning با آموزش embedding های قابل یادگیری در ورودی، Adapter با اضافه کردن شبکه های کوچک Bottleneck در لایه های Transformer و LoRA با یادگیری یک آپدیت کم‌رتبه برای وزن ها این هدف را دنبال می‌کنند.

(ب) معمولاً در زمان آموزش با روش LoRA ماتریس B با مقدار صفر مقداردهی اولیه می‌شود. دلیل این کار را توضیح دهید.

با مقداردهی صفر به پارامتر B، مقدار $\Delta W = BA = 0$ خواهد شد که در نتیجه‌ی آن برای $W' = W_0$ خواهد شد که دقیقاً مشابه با رفتار Pretrained Model در ابتدای فرآیند یادگیری هست، همچنین به دلیل صفر بودن B، در اولین مراحل آموزش گرادینان مربوط به A صفر است و ابتدا تنها B به‌روزرسانی می‌شود. پس از تغییر B، هر دو ماتریس A و B در ادامه‌ی فرآیند آموزش یادگیری خواهند کرد.

(ج) فرض کنید یک معماری Transformer که فقط شامل یک بلوک Encoder داریم. اندازه بعد لایه‌های پنهان و امبدینگ‌ها برابر d است و همچنین h برابر تعداد head ها در بلوک Multi-Head Attention است و در لایه Feed-Forward سائز لایه پنهان ۴ برابر و سپس به اندازه خود باز می‌گردد. اگر بخواهیم روش LoRA با رنک r را بر روی تمامی ماتریس‌ها اعمال کنیم، تعداد پارامترهایی که آموزش می‌بینند را بدست آورید. (فقط پارامترهای Feed-Forward و Attention را در نظر بگیرید و از بایاس‌ها صرف نظر کنید.)

در Multi-Head Attention، ماتریس‌های W_Q ، W_K و W_V که در لایه‌های Q ، K و V استفاده می‌شوند، هر کدام $d \times d$ هستند. همچنین d برابر تعداد head ها در بلوک Multi-Head Attention است و r رنک پارامترهای A و B است. $A \in \mathbb{R}^{d \times r}$ و $B \in \mathbb{R}^{r \times d}$ پارامترهای A و B هستند. d برابر تعداد لایه‌های پنهان است و r رنک پارامترهای A و B است. d برابر تعداد لایه‌های پنهان است و r رنک پارامترهای A و B است. d برابر تعداد لایه‌های پنهان است و r رنک پارامترهای A و B است.

برای Feed Forward Fully Connected ما ابتدا از $4 \times d \times d$ پارامتر داریم.

$$w_1 \in \mathbb{R}^{4d \times d} \Rightarrow \underbrace{(4d \times r)}_B + \underbrace{(r \times d)}_A = 5dr$$

و سپس به عنوان مثال می‌گیریم

$$(d \times r) + (r \times 4d) = 5dr$$

$$8dr + 2(5dr) = \underline{\underline{18dr}}$$

سپس در کل

(د) در سال‌های اخیر با توجه به عملکرد خوب روش LoRA در تسک‌های مختلف، توجهات زیادی به این روش شده است و Extension های مختلفی از این روش ارائه شده است. با تحقیق و جستجو، ۴ مورد از این روش‌ها را پیدا کنید و توضیح مختصری در مورد آن‌ها و تفاوتشان با LoRA ارائه دهید. (هر مورد را نهایتاً در ۲ الی ۳ خط توضیح دهید.)

روش‌هایی مانند QLoRA، AdaLoRA و DoRA برای بهبود محدودیت‌های LoRA معرفی شده‌اند. QLoRA با Quantization وزن‌های مدل پایه، مصرف حافظه را کاهش می‌دهد و امکان Fine-Tuning مدل‌های بزرگ‌تر را فراهم می‌کند. AdaLoRA با تخصیص تطبیقی Rank به لایه‌های مختلف، تعداد پارامترهای آموزش‌پذیر را به صورت بهینه توزیع می‌کند. DoRA نیز با تجزیه وزن به مؤلفه‌های Magnitude و Direction، محدودیت LoRA در تغییر وزن‌ها را کاهش داده و عملکردی نزدیک‌تر به Full Fine-Tuning ارائه می‌دهد.

روش	ایده اصلی	تفاوت با LoRA
LoRA	یادگیری $(\Delta W = BA)$ کم‌رتبه	روش پایه
QLoRA	LoRA + Quantization	کاهش حافظه با کم‌دقت کردن مدل پایه
AdaLoRA	Rank تطبیقی	تخصیص هوشمند پارامترها بین لایه‌ها
DoRA	تجزیه وزن به Magnitude و Direction	نزدیک‌تر شدن به Full Fine-Tuning

پرسش ششم (امتیازی - ۳۰ نمره)

ما این بخش را با مرور مختصری از فرمول‌بندی خود-توجه (Self-Attention) و کانولوشن‌ها آغاز می‌کنیم. هر دو روش به‌طور جداگانه و مشترک در وظایف مختلف پردازش زبان طبیعی استفاده شده‌اند. هدف ما از این سوال این است که درباره شباهت‌هایی که بین آن‌ها وجود دارد، بحث کنیم.

یک لایه خود-توجه در یک شبکه عصبی این گونه تعریف می‌شود که ورودی، ماتریس $Z \in \mathbb{R}^{N \times D_i}$ (نمایش‌دهنده N توکن ورودی) را می‌گیرد و ماتریس خروجی $O \in \mathbb{R}^{N \times D_o}$ را طبق عملیات زیر تولید می‌کند:

$$O = \text{Self-Attention}(Z) := \text{softmax} \left(\frac{ZW_q W_k^T Z^T}{\sqrt{d_k}} \right) ZW_v \quad (۱)$$

که در آن $W_v \in \mathbb{R}^{(D_i \times D_o)}$ ، $W_k \in \mathbb{R}^{(D_i \times D_{hid})}$ ، $W_q \in \mathbb{R}^{(D_i \times D_{hid})}$ که در آن ترتیب برای کوئری (query)، کلید (key) و مقدار (value) می‌باشند. این شکل خاص از توجه، به‌عنوان خود-توجه شناخته می‌شود، زیرا تمام ماتریس‌های وزن با همان ماتریس ورودی Z ضرب می‌شوند. برای وضوح بیشتر، ماتریس توجه $A \in \mathbb{R}^{n \times n}$ را به‌صورت زیر تعریف می‌کنیم:

$$A := ZW_q W_k^T Z^T \quad (۲)$$

عناصر این ماتریس به عنوان نمرات توجه (attention scores) شناخته می‌شوند و نشان می‌دهند که هر توکن چقدر به هر توکن دیگر مرتبط است. هر عنصر $\text{softmax}(A)$ را نیز احتمال‌های توجه (attention probabilities) می‌نامیم. به طور تجربی ثابت شده است که انجام مکانیسم خود-توجه بر روی سرهای متعدد مفید است. به عبارت دیگر چند سر توجه داشتن به این معنی است که چندین بار به صورت موازی در یک لایه، هر بار با ماتریس‌های وزنی مختلف، فرایند خود-توجه را انجام دهیم. سپس خروجی‌های سرهای توجه به صورت زیر ترکیب شوند:

$$\text{MultiHead-Self-Attention}(Z)_{t,:} = \left(\text{concat}_{h \in [N_h]} \left(\text{Self-Attention}_h(Z) W_h^{\text{out}} \right) \right)_{t,:} + b_{\text{out}} \quad (3)$$

که در آن $W_{\text{out}} \in \mathbb{R}^{(N_h \cdot D_o) \times D_{\text{out}}}$ و $b_{\text{out}} \in \mathbb{R}^{1 \times D_{\text{out}}}$ پارامترهای قابل آموزش هستند و N_h تعداد سرها است. ردیف‌های ماتریس ورودی Z معمولاً تعبیه‌های توکن‌ها هستند که توکن را در یک فضای برداری نشان می‌دهند. با این حال، اخیراً از لایه‌های توجه با پیچ تصویری به عنوان ورودی استفاده می‌شود که نتایج خوبی در وظایف بینایی رایانه مانند طبقه‌بندی تصویر به دست می‌آورد. در این حالت، تصویر ورودی با یک تانسور $Z \in \mathbb{R}^{(W \times H \times D_i)}$ کدگذاری می‌شود، جایی که W و H اندازه‌های تصویر ورودی هستند و D_i تعداد کانال‌ها است. ما می‌خواهیم فرمول‌بندی خود-توجه در معادله ۱ را طوری تطبیق دهیم که با پیکسل‌ها $p = (i, j)$ به جای توکن‌های کلمه کار کند. برای ساده‌سازی نوشتار، ماتریس‌ها را با $Z_{p,:}$ به عنوان $Z_{i,j,:}$ نمایان می‌کنیم.

(الف)

چند پارامتر قابل آموزش در یک لایه خود-توجه چندسر با N_h سر، مانند آنچه در معادله ۳ توضیح داده شده، وجود دارد؟

برای پاسخ سوال: اندازه $H \times W$ با سائز پیچ، $P \times P$ قسم سبب، سبب $\frac{H \times W}{P}$ ، اگر این مقدار N توکن در نظر بگیریم، در تمام لایه N توکن $P \times P \times C$ (تعداد کانال‌ها) ورودی و در لایه‌های دیگر اگر D_i در نظر بگیریم در نهایت به $Z \in \mathbb{R}^{N \times D_i}$ می‌رسیم. کانال‌ها $w \times h$ تعداد روز Patch $\leftarrow \text{تعداد Patch}$ $O(N^2 D_h)$

اما نکته آخر جدول، با افزایش H و W (وضعیت قدری میزان محاسبه Attention بیشتر می‌شود، اما تعداد پارامترها در Patch تغییر نمی‌کند. اینها w_k و w_v و w_q و w_o هستند. جدول براساس پیچ محاسبه اندازه P Patch، اما، حالاً محاسبه و ضرب و قسم بهترین اند.

(ب)

معادله ۱ و معادله ۳ را برای تک پیکسل کوثری q بازنویسی کنید، به طوری که خود توجه به صورت زیر بدست آید:

$$\text{Self-Attention}(Z)_{q,:} \in \mathbb{R}^{(1 \times D_o)}$$

و

$$\text{MultiHead-Self-Attention}(Z)_{q,:} \in \mathbb{R}^{(1 \times D_{out})}$$

ویژگی خاص خود-توجه همانطور که در معادله ۱ تعریف شده است، این است که تغییرات در ترتیب را تحمل می‌کند (permutation invariant). با این حال، هم زمانی که با دنباله‌های متنی کار می‌کنیم و هم زمانی که با تصاویر کار می‌کنیم، موقعیت هر توکن یا پیکسل اطلاعات مهمی را حمل می‌کند که می‌خواهیم آن را حفظ کنیم. برای انجام این کار، ما از یک ماتریس موقعیت (ماتریس $P \in \mathbb{R}^{N \times D_i}$ که می‌تواند یادگرفته شده یا ثابت باشد) استفاده می‌کنیم که فرم ماتریس توجه A در معادله ۲ را به صورت زیر تغییر می‌دهد:

$$A := (Z + P)W_q W_k^T (Z + P)^T \quad (۴)$$

رمزگذاری‌های موقعیتی می‌توانند مطلق (absolute) یا نسبی (relative) باشند. در رمزگذاری‌های مطلق، یک بردار موقعیت P_p برای پیکسل ورودی p ، تنها به موقعیت مطلق p در ورودی بستگی دارد. بنابراین، ما می‌توانیم معادله ۴ را برای پیکسل کوثری q و پیکسل کلید k گسترش دهیم و به شکل زیر برسیم:

$$\begin{cases} A_{q,k}^{\text{absolute}} = (Z_{q,:} + P_{q,:})W_q W_k^T (Z_{k,:} + P_{k,:})^T \\ = Z_{q,:}W_q W_k^T Z_{k,:}^T + Z_{q,:}W_q W_k^T P_{k,:}^T + P_{q,:}W_q W_k^T Z_{k,:} + P_{q,:}W_q W_k^T P_{k,:} \end{cases} \quad (۵)$$

از طرف دیگر، در رمزگذاری نسبی، بردار موقعیت برای پیکسل کوثری تنها به تفاوت موقعیت نسبی $\delta := q - k$ بستگی دارد. $(\delta_1, \delta_2) \in \mathbb{Z}^2$

حالت مطلق

$$A_{q,k}^{\text{relative}} := Z_{q,:}W_q W_k^T Z_{k,:}^T + Z_{q,:}W_q \widetilde{W}_k r_\delta + u^T W_k Z_{k,:} + v^T \widetilde{W}_k r_\delta \quad (۶)$$

در اینجا، u و v بردارهای قابل یادگیری هستند و ما یک ماتریس پارامتر جدید $\widetilde{W}_k \neq W_k$ معرفی می‌کنیم. در حالی که انواع مختلفی از رمزگذاری‌های نسبی وجود دارد، در این سوال ما به یک فرم خاص از رمزگذاری نسبی به نام رمزگذاری گاوسی می‌پردازیم که از معادلات زیر تبعیت می‌کند:

$$v^{(h)} := -\alpha^{(h)} \begin{pmatrix} 1 \\ -2\Delta_1^{(h)} \\ -2\Delta_2^{(h)} \end{pmatrix}, \quad r_\delta := \begin{pmatrix} \|\delta\|^2 \\ \delta_1 \\ \delta_2 \end{pmatrix}, \quad W_q = W_k := 0, \quad \widetilde{W}_k := I \quad (۷)$$

ما D_p را به عنوان بُعد هر دو $v^{(h)}$ و r_δ در نظر می‌گیریم. در این حالت، داریم $D_p = 3$. همچنین، $\Delta_1^{(h)}$ و $\Delta_2^{(h)}$ پارامترهای قابل یادگیری هستند.

معادله ۶ را ساده کنید

(ج)

عبارات داده شده در معادله ۷ را در معادله ۶ قرار دهید تا $A_{q,k}^{\text{relative}}$ را ساده کنید.

$$\begin{aligned} A_{q,k}^{\text{relative}} &= \begin{bmatrix} -\alpha^{(h)} & 2\alpha^{(h)}\Delta_1^{(h)} & 2\alpha^{(h)}\Delta_2^{(h)} \end{bmatrix} \begin{bmatrix} \|\delta\|^2 \\ \delta_1 \\ \delta_2 \end{bmatrix} = -\alpha^{(h)}\|\delta\|^2 + 2\alpha^{(h)}\Delta_1^{(h)}\delta_1 + 2\alpha^{(h)}\Delta_2^{(h)}\delta_2 \\ &= -(\alpha^{(h)}\|\delta\|^2 - 2\alpha^{(h)}\Delta^{(h)}\delta) \end{aligned}$$

با فرض نرمش w_2 و $w_k = 0$ ، میزان توجه به معطوف؟ ما حالا می‌توانیم بگوییم که
و اما D_o شود، Attention در این حالت خاص! δ بیشتر به توجه می‌دهد.

(د)

هزینه محاسباتی یک لایه توجه تک‌سر با استفاده از رمزگذاری مطلق چقدر است؟ فرض کنید که P قبلاً محاسبه شده است. هزینه محاسباتی چه خواهد بود اگر لایه توجه از رمزگذاری گاوسی استفاده کند؟ از نماد O بزرگ برای مقایسه دو حالت نسبت به $D_p \gg D_o = D_{hid} = D_i = D_x$ و N استفاده کنید.

برای ورود جدید (الف)

$$X = Z + P \Rightarrow O(N D_x) \quad \text{هر کدام} \quad \underbrace{(N \times D_x)}_X \underbrace{(D_x \times D_x)}_{w_{k,q,v}} = N D_x^2 \Rightarrow 3(N D_x^2)$$

و برای ماتریس Attention

$$= (N \times D_x) (D_x \times N) = N^2 D_x$$

در حالت گاوسی هر عنصر برای موقعیت مکانی رابطه $r_i \in \mathbb{R}^P$ ساخته می‌شود که بسیار کم بعد تر از P است. ۱. برای این حالت مقدار $N^2 D_p$ برای مجموع جفت‌ها را می‌توانیم محاسبه کنیم. ۲. برای $\bar{v}$ نیز به P کار می‌کنیم و اینجا هم D_p کار می‌کند. $O(N D_o)$

و بنابراین با جمع هزینه برای این حالت:

$$O(N D_x^2 + N^2 D_x + N^2 D_p)$$

با این توضیحات
۱. هر عنصر
۲. اضافه می‌شود

و بعد از آن $O(N D_x^2 + N^2 D_x)$

۳. طوری که هر دو $O(N^2)$ می‌توانند
۴. هزینه گاوسی $\gg$ بزرگ می‌شود

Softmax (Gaussian $\times \bar{v}$)
($N \times N$) ($N \times D_x$)

ما اکنون به طور مختصر ساختار یک کانولوشن را مرور می‌کنیم. با در نظر گرفتن یک تانسور تصویر $Z \in \mathbb{R}^{W \times H \times C_{in}}$ یک ماتریس وزن $W \in \mathbb{R}^{K \times K \times C_{in} \times C_{out}}$ و یک بردار بایاس $b \in \mathbb{R}^{C_{out}}$ که در آن K اندازه کرنل و C_{in} و C_{out} به ترتیب تعداد کانال‌های ورودی و خروجی هستند، خروجی یک لایه کانولوشنی برای یک پیکسل منفرد $p = (i, j)$ به صورت زیر خواهد بود:

هر پیکسل در i, j به همراه هار خود نشانه می‌دهد

$$\text{Convolution}(Z)_{i,j,:} := \left(\sum_{(\delta_1, \delta_2) \in \Delta_K} Z_{i+\delta_1, j+\delta_2,:} W_{\delta_1, \delta_2,:} \right) + b \quad (8)$$

انرژی کرنل

که در آن Δ_K مجموعه‌ای از تمام جابجایی‌های ممکن است که توسط کرنل کانولوشنی با اندازه K مجاز است.

$$\Delta_K := \left\{ \left[\left\lfloor -\frac{K}{2} \right\rfloor, \dots, \left\lfloor \frac{K-1}{2} \right\rfloor \right] \times \left[\left\lfloor -\frac{K}{2} \right\rfloor, \dots, \left\lfloor \frac{K-1}{2} \right\rfloor \right] \right\} \quad (9)$$

در نهایت، هدف ما اثبات قضیه‌ای است که شباهت‌های کلیدی میان قابلیت‌های محاسباتی لایه‌های خودتوجهی و لایه‌های کانولوشنی در یک معماری عصبی نمایان می‌سازد. قضیه اصلی: یک لایه خود-توجه چند-سر که روی K^2 سر با ابعاد D_o و ابعاد خروجی D_{out} عمل می‌کند، با استفاده از یک رمزگذاری موقعیتی نسبی با ابعاد $D_p \geq 3$ ، می‌تواند وظیفه هر لایه کانولوشنی با کرنل اندازه $K \times K$ و کانال‌های خروجی D_{out} را ادا کند.

ما اثبات این قضیه اصلی را به اثبات جداگانه دو قضیه فرعی تقسیم می‌کنیم. قضیه ۱ به شرح زیر است: قضیه ۱: با فرض اینکه یک لایه خود-توجه چند-سر با $N_h = K^2$ سر و $D_o \geq D_{out}$ داده شده باشد، فرض کنیم که $f: [N_h] \rightarrow \Delta_K$ یک نگاشت یک‌به‌یک بین سرها و جابجایی‌ها است. همچنین فرض می‌کنیم که برای هر سر داریم:

فاصله
اگر h و $k > 0$ بود
در لایه خودتوجهی
اعمال

$$\text{softmax}(A_{q,:}^{(h)})_k = \begin{cases} 1 & \text{اگر } f(h) = q - k \\ 0 & \text{در غیر این صورت.} \end{cases} \quad (10)$$

آنگاه برای هر لایه کانولوشنی با کرنل $K \times K$ و کانال‌های خروجی D_{out} ، یک مجموعه از وزن‌ها $\{W_v^{(h)}\}_{h \in N_h}$ وجود دارد که به گونه‌ای است که:

هر Head مستقل به موقعیت
kernel

(5)

معادله ۳ را می‌توان به صورت زیر نوشت:

$$\text{MultiHead-Self-Attention}(Z) = \sum_{h \in [N_h]} \text{softmax}(A^{(h)}) Z W^{(h)} + b_{out} \quad (11)$$

وزن $W^{(h)}$ را به صورت تابعی از W_v و W_{out} بیان کنید. سپس از $W^{(h)}$ برای نوشتن یک عبارت برای

$$\text{MultiHead-Self-Attention}(Z)_{q,:} \in \mathbb{R}^{D_{out}}$$

استفاده کنید.

چون می‌توانیم به Convolution و Attention از این سر جدا کنیم
Head هست به هر سر متعلق به یک Head =
هستند $W_0^{(1)}, W_0^{(2)}, W_0^{(3)}, \dots, W_0^{(N_h)}$

$w_0^{(h)}$
 Head هر $\hookrightarrow$
 $A^{(h)} = \frac{QK^T}{\sqrt{d_h}} \Rightarrow H_{(h)} = \text{Softmax}(A^{(h)}) Z W_V^{(h)} \times W_O^{(h)}$
 مزوجی میان Head هر
 $\Rightarrow W_{Total}^{(h)} = W_O^{(h)} \times W_V^{(h)}$ $\hookrightarrow$ Head هر دین
 $\Rightarrow \sum_h \text{Softmax}(A^{(h)}) Z W_{Total}^{(h)} + b_{out}$
 11 بهر نام Head ها

(9)

از مفروضات قضیه ۱ استفاده کنید تا $\text{MultiHead-Self-Attention}(Z)_{q,:}$ را به شکلی بیان کنید که معادل یک لایه کانولوشنال طبق تعریف معادله ۸ باشد.

$$\text{Convolution}(Z)_{i,j,:} := \left(\sum_{(\delta_1, \delta_2) \in \Delta_K} Z_{i+\delta_1, j+\delta_2,:} W_{\delta_1, \delta_2,:} \right) + b \quad (8)$$

$$\text{MultiHead-Self-Attention}(Z) = \sum_{h \in [N_h]} \text{softmax}(A^{(h)}) Z W^{(h)} + b_{out} \quad (11)$$

از طرف
دانش

$$\text{softmax}(A_{q,:}^{(h)})_k = \begin{cases} 1 & \text{اگر } f(h) = q - k \\ 0 & \text{در غیر این صورت.} \end{cases}$$

$\text{Multihead-Self-Attention}(Z)_{q,:} = \sum_{h=1}^{N_h} \text{Softmax}(A_{f(h)}) Z W^{(h)} + b_{out}$
 اگر $f(h)$ را به هر q مقابله شود h نام هر فرکانس $f(h)$ و $W^{(h)}$ قدرت دارد

(12)

حد زیر را برای هر دو حالت $\delta = \Delta$ و $\delta \neq \Delta$ حل کنید و توضیح دهید که چرا این معادله ۱۰ را برآورده می‌کند:

معادله ۱۰

$$\lim_{\alpha \rightarrow +\infty} \text{softmax}(A_{q,:})_k \quad (13)$$

بر اساس در رابطه

$$\text{softmax}(A_{q,:}^{(h)})_k = \begin{cases} 1 & \text{اگر } f(h) = q - k \\ 0 & \text{در غیر این صورت.} \end{cases}$$

$$c = \sum_{i=1}^n v_i \alpha_i \quad (1)$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} \quad (2)$$

اگر متغیر c رو به دست جمع وزن در بسیم و اگر

با استفاده از رابطه ۱۲، مینیمم هزینه طریقه‌ی بهینه مقدار α را پیدا می‌کنیم و مینیمم هزینه طریقه‌ی بهینه α را در رابطه ۱۱ قرار می‌دهیم و نتیجه‌ی بهینه α را به دست می‌آوریم.

attention معمولی

حالت گاوسی

دمای α حالت محوری فقط α ، از $\|\delta - \Delta\|^2 - \alpha$ استفاده می‌شود که به این عبارت α می‌گویند.

(ح)

از تعریف رمزگذاری گاوسی استفاده کنید و تعیین کنید که برای کدام مقدار ثابت c مفروضه در معادله ۱۲ برقرار است. با ترکیب قضیه ۱ و قضیه ۲، ما قضیه اصلی را اثبات کردیم و نشان دادیم که تحت برخی شرایط، یک لایه توجه می‌تواند یاد بگیرد که مانند یک لایه کانولوشنال رفتار کند. در نهایت، بررسی کنید آیا این نتیجه برای هاپیرپارامترهای مختلفی که معمولاً در هنگام بهینه‌سازی شبکه‌های کانولوشنی تنظیم میشوند صادق است یا خیر.

$$A_{q,k} = -\alpha(\|\delta - \Delta\|^2 + c) \quad (12)$$

قسمت اول سؤال: با توجه به رابطه ی ۱۲ و همچنین رابطه $\text{softmax}(A)$ ، با به توان رسیدن عبارت $\frac{-\alpha c}{e} = e^{\frac{-\alpha(\|\delta - \Delta\|^2 + c)}{e}}$ مقدار e

، با صورت و منخرج به اضای مقادیر مختلف حذف می‌شود، بنابراین به اضای تمامی مقادیر c برقرار خواهد بود رابطه.

قسمت دوم: این روابط فقط تحت فرض‌های خاص اثبات شده است و لزوماً برای تمام تنظیمات رایج شبکه‌های کانولوشنی مانند Stride، Padding، Dilation و Pooling برقرار نیست؛ بنابراین نمی‌توان آن را به همه‌ی Hyperparameterهای CNN تعمیم داد.

(ط)

برای کدام اندازه‌ها و مقادیر پدینگ خروجی یک لایه خود-توجه چندسر معادل خروجی یک لایه کانولوشنال است؟

(ی)

آیا یک لایه خود-توجه چندسر می‌تواند یک کانولوشن با dilations دلخواه را با وجود محدودیت‌های ساختاری مرتبط با تصویر ورودی بیان کند؟ دلیل خود را توضیح دهید.

(ک)

آیا یک لایه خود-توجه چندسر می‌تواند یاد بگیرد که یک کانولوشن با stride را شبیه‌سازی کند؟ اگر بله، چرا؟ در غیر این صورت، چرا این امکان وجود ندارد؟

ط) معادل بودن با Convolution فقط زمانی برقرار است که نحوه‌ی مدیریت مرزهای تصویر (Boundary Handling) در Attention و Convolution یکسان باشد. خود Self-Attention ابعاد خروجی را تغییر نمی‌دهد؛ بنابراین برای شبیه‌سازی Convolution با Padding‌های مختلف، باید توکن‌های خارج از تصویر به همان روش CNN (مثلاً Zero Padding یا حذف در حالت Valid) مدیریت شوند.

(ی) *دلیل dilation مدیریت به یکس لایه‌ها با اندازه 5 حاصل می‌شود، این حاصل می‌شود در تمام 160 کار حاصل می‌شود 115-81² توکن‌ها اکل می‌شود بنابراین این نقشه 96 می‌شود و این 96 می‌شود.*

(ک) *در صورت طبیعی، هرگاه که ما از Attention، هیچ توکنی نداریم و فقط می‌شود که همه توکن‌ها وزن 1 داشته باشند. توکن‌ها 96 می‌شوند و این 96 می‌شود و این 96 می‌شود. Attention این کار می‌کند.*